

Cryptocurrency Investment Product Model Design and Implementation Report

Event-Driven Rebalancing Based on Sentiment Spikes

Station #3: Model Design and Sentiment Analysis

a) Model Selection and Design

Factor Creation

An extensive suite of quantitative factors was engineered from the raw OHLCV data to capture correlations between factors to minimise risk and maximise returns, and detect unusual trading activity. These factors include

- Return based factors (e.g. `return_2w`, `return_4w`)
- Momentum based factors (e.g. `momentum_14`, `momentum_90`)
- Risk factors (e.g. `volatility_14`, value at risk 5% (`VaR_5`), Max Drawdown)
- Volume based factors (e.g. `usd_v_7d`)

After news analysis in station 2, the `sentiment_index` column was added for each specific cryptocurrency as a primary factor, for overall mood of news articles related to each coin.

Sentiment Analysis and Indexing

A dual-model approach was implemented for robust sentiment scoring, combining a lexicon-based model with a modern transformer-based model:

- Valence Aware Dictionary and sEntiment Reasoner (VADER): utilised for its speed and interpretability, to improve accuracy on financial news
- Hugging Face Transformer Model: utilised for advanced deep contextual understanding of news text and state-of-the-art performance
- The final sentiment score `comp_avg` averages the outputs from both models, for simplicity.

Sentiment indicators were created from news analysis. This included a `sentiment_index` for each cryptocurrency, which was used as a direct model feature aggregating all relevant per-article sentiment scores, and a separate `market_sentiment_index` for the overall market mood, which was used purely for comparative analysis in the final visualisations.

Optimisation and Strategy Models

Rather than traditional mean-variance portfolio optimisation, a factor-based signal generation approach was used instead. A suite of Machine Learning (regression) models were trained to predict future returns based on the created factors. These include

- Ordinary Least Squares (OLS)
- Ridge Regression (OLS with L2 regularisation)
- Gradient Boosting Regressor (powerful non-linear ensemble model)

A naive Equal Weight (EW) strategy was also included, assuming a constant “buy” signal for all assets. This serves as the essential benchmark to evaluate machine learning model performance.

Combining Signals, Dynamic Rebalancing and Constraints

The final trading signal was designed as a hybrid, combining the predictive power of the models with the averaged sentiment score. The core logic of generating a “buy” signal was:

`(predicted_return > 0) AND (sentiment_index > threshold)`

This ensures the strategy only enters positions when both the models and the news sentiment are positive. Signals are generated on a daily basis, implying daily rebalancing frequency.

Critical factors include transaction costs, slippage and trading constraints, as these were not incorporated into the backtest for this project and thus should be included in a production environment.

b) Sentiment Analysis Implementation

The text processing pipeline from station 2 involved cleaning (HTML removal and lowercasing), tokenisation, stop-word removal (removal of irrelevant words like “the”) and lemmatisation, to prepare news text for analysis.

VADER Sentiment Model

VADER was implemented using the `nltk.sentiment.vader` library. A custom dictionary (`vader_custom_lexicon.py`) was created to adapt VADER to cryptocurrency-specific terms (e.g., 'bullish', 'bearish', 'HODL'), by matching these terms to pre-defined sentiment scores. This helps improve domain-specific accuracy.

Hugging Face API Model

The `mrm8488/distilroberta-finetuned-financial-news-sentiment-analysis` model was selected for its state-of-the-art performance, as it was specifically fine tuned on financial news text, and was a trustworthy model with over 230k downloads. Python's requests library was used to query the Hugging Face Inference API, with robust error handling to fall back to VADER if the API call fails.

Long articles were split into smaller, 400-word chunks, before being passed into the model, and then chunk sentiment scores were averaged to produce a final score for each article. This was necessary, as the model contains token limits. The model's output probabilities for 'positive' and 'negative' labels were converted into a single compound score in the [-1, 1] range.

Lastly, the compound score from this model and the VADER model was averaged to produce the final per-article `comp_avg` score, creating a more reliable and nuanced sentiment signal.

Sentiment Index Construction

The primary method for aggregation was a simple daily mean. For each coin (and for the MARKET symbol), all per-article `comp_avg` scores for a given day were averaged to produce a single `sentiment_index` for that day. This effectively normalises for variations in news volume, preventing a single highly positive or negative article from dominating.

Integration with Portfolio models

The coin specific `sentiment_index` was used in two ways

1. As a direct input feature for each of the regression models (OLS, Ridge, Gradient Boosting)
2. As a final filter in the trading signal logic, adding a layer of confirmation to the raw sentiment

The `markets_sentiment_index` was intentionally kept separate from the modelling pipeline. This was to avoid overweighting macro trends, and thus was only used for analytical plots to provide more contextual insights to users in station 4.

c) Model Assumptions

The following assumptions were made during the model design:

1. Market behaviour: the model assumes that crypto markets are not perfectly efficient, and exhibit patterns like momentum and sentiment effects that are, only to some extent, predictable.
2. Feature relationships: the model assumes a stable, learnable relationship exists between features, such as volatility and sentiment, and future relationships.
3. Predictive power of sentiment: collective sentiment, as captured in news articles, was assumed to be a leading indicator of price moments.
4. Transaction costs: the model assumes zero transaction costs and no slippage for the purpose of this prototype.
5. Capital allocation: the model assumes an equal amount of capital allocated to each "buy" signal generated by a strategy on any given day.

d) Technical Constraints and Limitations

There are also many technical constraints further limiting the model's feasibility:

1. API dependency: The sentiment scoring process relies on a third-party Hugging Face API, which introduces latency. Querying on years of news articles at once will take hours of runtime.
2. Data window: The analysis is constrained by the available historical news data, which may not be sufficient to capture all market regimes.
3. VADER limitations: VADER struggles with complex financial jargon, sarcasm, and nuanced statements that transformer models are better equipped to handle. It also fails to distinguish between coin symbols, such as NEAR, which also correspond to a common English word.
4. Real-time challenges: The current batch-processing pipeline is not designed for real-time signal generation, which would require a more robust streaming architecture

5. Optimisation approach: The model predicts returns but does not perform actual portfolio weight optimisation (e.g. maximising a Sharpe Ratio). It acts as a signal generator for an equal-weighting scheme.

e) Expected Implementation Results

It is expected that the sentiment-aware strategies, particularly the Gradient Boosting regressor which is more complex, will outperform the Equal Weight (EW) benchmark on a risk-adjusted basis. This can be proven through the All Strategies VS Buy & Hold line chart (see *Appendix 1*).

The sentiment based strategies are expected to exhibit higher Sharpe and Sortino ratios, suggesting better returns for the risk taken. The EW benchmark is expected to have higher volatility. This can be shown in both the Strategy Performance Comparison radar chart (see *Appendix 1*) and the Strategy Performance Summary table (see *Appendix 2*).

Lastly, a positive correlation is expected between sentiment lag features and asset returns, which would validate the model's core hypothesis.

f) Model Boundaries and Risks

There are also many boundaries and risks associated with the model:

1. Market conditions: The strategies must underperform in "risk-off" or "black swan" market conditions where all assets are highly correlated and sentiment is universally negative.
2. Time horizons: The predictive power of sentiment is assumed to be short term.
3. External factors: The model cannot account for sudden, impactful events like exchange hacks, major regulatory shifts, or deliberately manipulated "fake news"
4. News sourcing and bias: The model is susceptible to any biases in the news sources it consumes. It also only consumes news articles from a single API (CoinDesk), ignoring social media posts and other crypto media providers.

g) Station #3 Definition

From a Data Management Perspective, Station #3 is the "Transformation and Modelling Hub." It consumes cleaned time-series data (prices, volume) and unstructured text data (news) from the preceding stations, and applies machine learning models (VADER or hugging face, then OLS, RIDGE or Gradient Boosting regression) to transform this data into a structured set of predictive outputs—namely, daily trading signals and performance metrics—which are then passed to the implementation and visualisation station.

Station #4: Model Implementation

Access the frontend at the following URL: <https://eloquent-sprinkles-0ad32e.netlify.app/>

a) Product Design

TILT.ai is a robust and intuitive front-end prototype built with React.js. It serves as a dashboard for visualising the outputs of the sentiment analysis and portfolio optimisation strategies, designed to be easy to use for beginners in crypto. The primary target users are tech-savvy retail crypto investors looking for data-driven insights beyond social media.

Currently the prototype is static, but for a production rollout it is expected that this product render components dynamically. *Screenshots of these pages can be found in Appendix 3.*

Key features and functionality:

- Main dashboard: A grid-based overview of all key market and strategy performance charts
- Sentiment indicators: visualisations including per-coin Fear & Greed gauges, bar charts comparing returns, volatility and coin sentiment to the market, and time series plots of sentiment
- Strategy comparison: Dedicated pages and charts including cumulative returns, heat maps and performance radar charts to compare the performance of each portfolio optimisation strategy
- News feed: A dedicated section for browsing recent crypto news articles, showing their sentiment score
- Chatbot: currently a placeholder, the UI element future-proofs integration with an AI-powered assistant to give users more crypto insights in a secure manner

b) Implementation Steps

Below lists the entire implementation process (including those outlined in station 3):

1. Sentiment analysis implementation: VADER and Hugging face models were implemented in Python to process clean news text and generate per-article sentiment scores.
2. Sentiment index construction: the per-article scores were aggregated using pandas to create daily time series for each coin and the overall market.
3. Integration and model training and validation: sentiment indices were merged with OHLCV data and used as features to train a suite of regression models (OLS, Ridge, Gradient Boosting) to predict future returns.
4. Backtesting framework development: a simple backtesting framework was implemented by calculating daily returns of each strategy based on the generated trading signals.
5. Output generation: the Python backend was finalised to output all necessary plots as static image files and all key data tables as .txt files.
6. File migration for frontend access: the outputted files were moved to Cloudinary, so they can be accessed via public URLs for use in the next step.
7. Application development: a multi-page web application was developed using React.js and styled with Tailwind CSS. `react-router-dom` was used for routing and navigation.

8. Version control: the web application was managed in GitHub to ensure efficient version control during development. GitHub integration here also helped with automated deployment scripts.
9. Conversational AI integration: Google Gemini was used as an interactive development partner to:
 - write, debug and iteratively refactor the Python data analysis and plotting scripts,
 - generate the majority of the React.js application structure, including components, pages, routing and styling, from a detailed prompt, and
 - assist in solving technical challenges related to both of the above, such as model design approaches, syntactical errors and deployment steps.

c) Implementation Challenges

Model performance

Initial versions of the linear models, specifically OLS and Ridge, failed to generate any trading signals. The issue was due to the fact that the models' predictions were too conservative to cross the signal threshold. This was solved by refactoring the strategy logic to be quantile-based, ensuring signals were generated for the top-performing assets regardless of the absolute prediction value.

Data pipeline bugs

A recurring `MergeError` in pandas was traced to a data type mismatch between `datetime64` and object date columns. The solution was to ensure date columns in all pandas DataFrames were explicitly standardised to `datetime64` before any merge operations.

Front-end testing and deployment

Initial attempts to run the frontend in a developer environment using Vite resulted in persistent localhost connection errors. The project's entire build system had to be refactored away from Vite to use the more established react-scripts (made using the `create-react-app` library).

Deploying the multi-page application to a basic static host, GitHub pages, caused 404 errors on page refreshes. Nothing was rendering on the screen, which was likely due to the fact that GitHub pages was only limited to one deployed project app per user (I already had another app deployed to GitHub pages). The solution was simply to migrate to Netlify (see *Appendix 4*), a more robust and simple modern web app deployment platform. A `_redirects` file was added to solve the routing issue, correctly handling all path requests and allowing the use of clean URLs with `BrowserRouter`.

UI/UX Requirements

Numerous iterative refinements were made to the frontend-end, including making plots responsive, adjusting gauge layouts, adding search bars, and fixing accessibility warnings in components that were blocking Netlify deployments.

d) Client Introduction Strategy

Introducing this sentiment-enhanced portfolio optimisation tool to clients should be carried out using the following recommended steps:

1. Pilot Testing: begin with a closed pilot program, allowing a select group of clients to use the platform with paper portfolios to gather feedback and validate results.
2. Risk assessment and crypto investor education: provide clear documentation and materials explaining the role of sentiment analysis, assumptions of the models, and inherent risks of quantitative strategies. Emphasise that this is a data driven tool, not a "get rich quick" scheme.
3. Demonstration of sentiment and historical value-add: use the interactive prototype to demonstrate the platform's value. The All strategies VS Buy & Hold line chart (see Appendix 1) is the most powerful tool for this, visually proving the potential outperformance of the sentiment-aware models.
4. Gradual rollout strategy: launch with capital limits and recommend clients allocate only small portion of their portfolio initially while they validate sentiment signals and performance.
5. Observability and site reliability: implement robust performance monitoring and reporting approaches, including alerts for data pipeline failures and dashboards for tracking key sentiment and strategy metrics.
6. Support and maintenance plan: establish dedicated support and maintenance plans for the market data and news processing systems that ensure data integrity, freshness and uptime.

e) Customer Journey

Below is the complete user experience:

1. Discovery: a user discovers TILT.ai via a search engine and arrives at the Landing page, where they can gain an overview of the platform's features and purpose.
2. Onboarding: the user signs up via the Login/Register page and is immediately taken to the dashboard. If logging in for the first time, they complete a simple onboarding process.
3. Daily usage: The user reviews the main Dashboard to get a high-level overview of the market sentiment and strategy performance. They check the latest sentiment scores and coin performance ranking tables
4. Decision making: the user navigates to the Strategies page to see the current strategy signals table, which shows which coins the models recommend buying today
5. Deeper analysis: the user clicks on the Prices page to deep dive into the sentiment and performance charts for a specific coin like BTC. Additionally, the user can click on the News page to view related news articles and their sentiment score
6. Account management: The user navigates to the Settings Page to manage their profile and notification preferences

Screenshots of these pages can be found in Appendix 3.

f) Conversational AI (Google Gemini) Integration

In this project, Google Gemini functioned as an expert pair programmer and interactive coding assistant. It was not integrated into the final product as a user-facing feature, but was integral to the development process.

Gemini was not used to generate entire Python scripts, but assisted in the majority of refactoring, exception handling and efficient coding techniques for data processing and

visualisation. Gemini was used more for generating entire components, pages and styling in the complete React.js front-end application.

The development was an iterative, conversational process. The following were provided as prompts:

- High level goals (e.g. "add a radar chart for all portfolio optimisation strategies...")
- Detailed prompts (e.g. for the entire React app)
- Debugging information (e.g. "fix the issue with the OLS model not returning any signals...")

Gemini then provided complete, refactored code, explanations and solutions. This approach was critical in accelerating the development lifecycle, from initial data analysis to a fully realised front-end prototype. It was particularly effective at solving complex problems (like the Netlify redirect issue) and implementing best data science and front-end engineering practices.

A detailed prompt for generating the entire React frontend app can be found in Appendix 5.

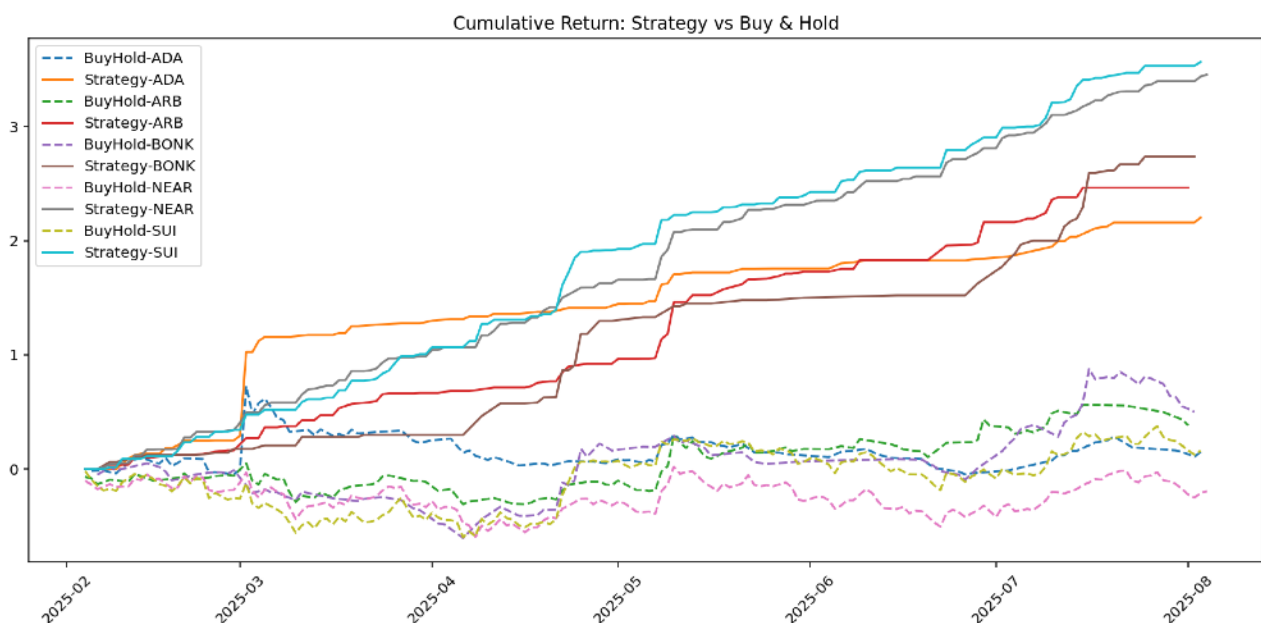
g) Station #4 Definition

From a Data Management Perspective, Station #4 is the "Presentation and Delivery Layer." It consumes the final structured data outputs, in the form of image visualisations and text tables, and presents them through a user-first application. This station is responsible for the entire user experience, translating complex back-end analytics into an intuitive, interactive, and visually appealing dashboard that allows users to consume and act upon the generated insights.

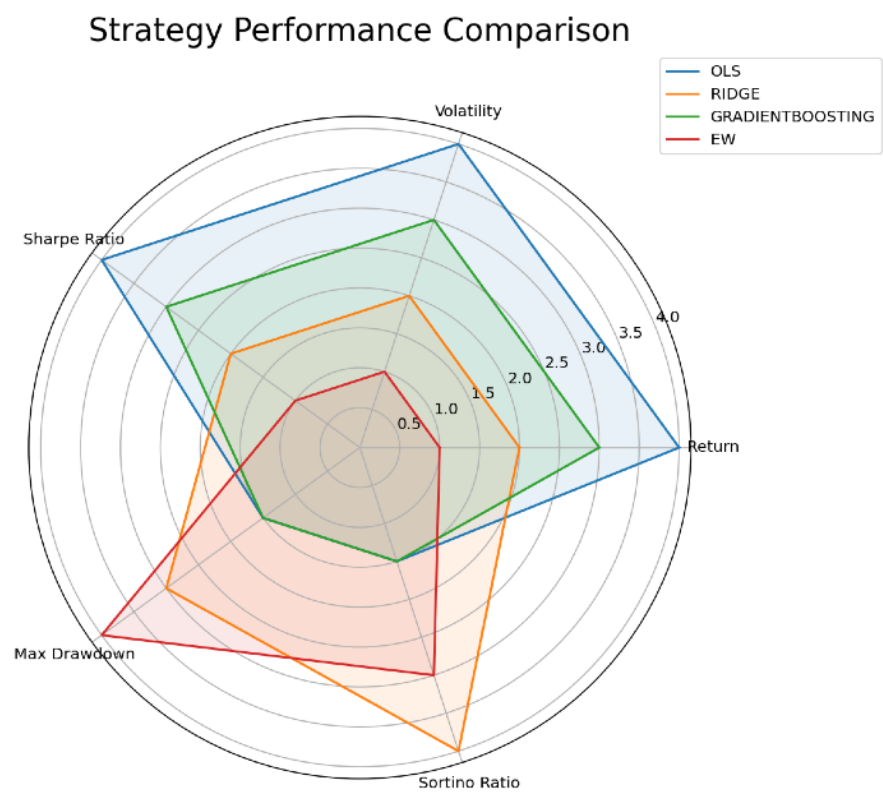
Appendix

Appendix 1 – Data charts/plots/visualisations

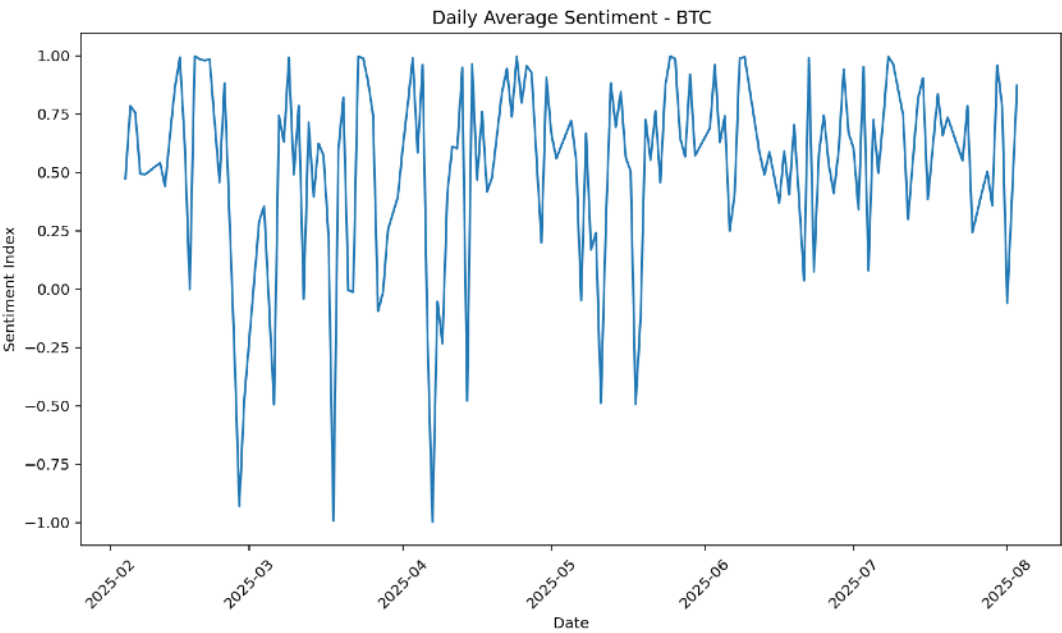
1A. All Strategies VS Buy & Hold for Top 5 Coins line chart



1B. Strategy Performance Comparison radar chart



1C. Daily Average Sentiment for BTC line chart



1D. Overall Market Sentiment Fear-Greed gauge



Appendix 2 – Data tables outputted as txt files

2A. Strategy Performance Summary table

Strategy	Return	Volatility	Sharpe Ratio	Max Drawdown	Sortino
Ratio					
OLS	6.092	0.450	13.541	0.000	0.000
RIDGE	6.029	0.451	13.380	-0.001	1800.948
GRADIENTBOOSTING	6.090	0.450	13.538	0.000	0.000
EW	0.415	0.758	0.547	-0.374	0.853

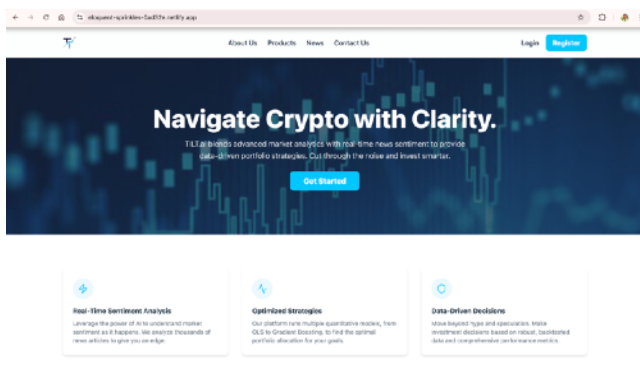
2B. Latest Sentiment Scores for Each Coin table

Symbol	LatestSentiment
AAVE	0.858
ADA	0.999
ALGO	0.970
APT	0.998
ARB	-0.549
ATOM	0.479
AVAX	0.997
BCH	-0.493
BGB	0.999
BNB	0.999
BONK	0.998
BTC	0.873
CRO	0.498
DOGE	0.914
ENA	0.951
ETC	0.637
ETH	0.814
GT	0.999
HBAR	0.990
HYPE	-0.978
ICP	0.998
KAS	0.998
LINK	0.759
LTC	0.998
MANTLE	0.735
NEAR	0.516

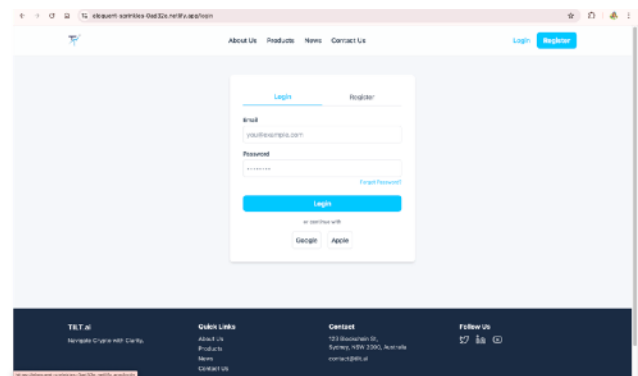
OKB	0.494
ONDO	0.768
PENGU	0.996
PI	0.997
SHIB	0.993
SOL	0.995
SUI	0.995
TAO	0.958
TON	0.739
TRX	0.840
VET	0.999
XLM	0.997
XMR	0.793
XRP	0.531
MARKET	-0.008

Appendix 3 – Frontend screenshots

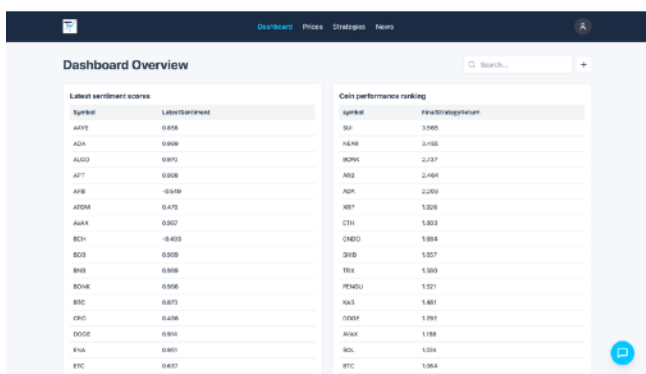
3A. Landing Page



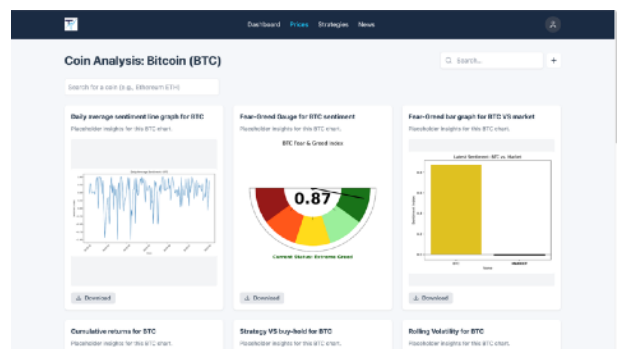
3B. Login/Register Page



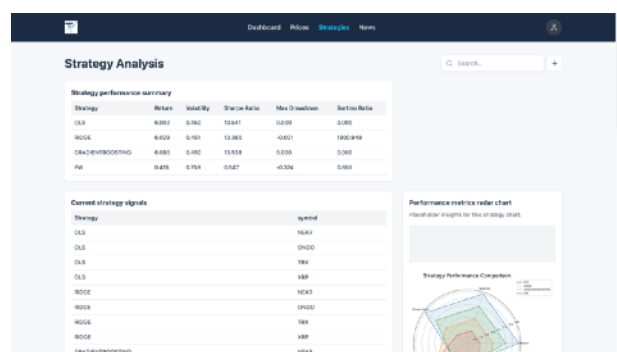
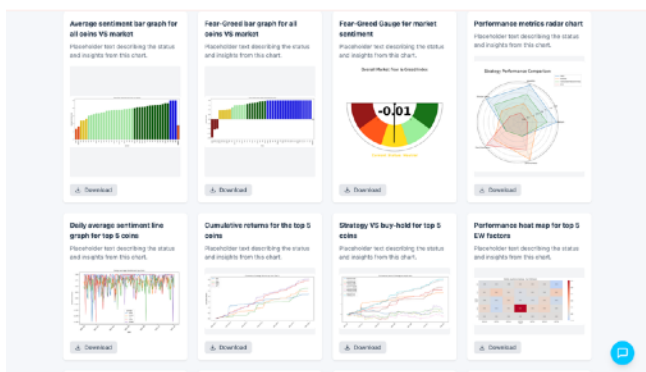
3C. Dashboard Page



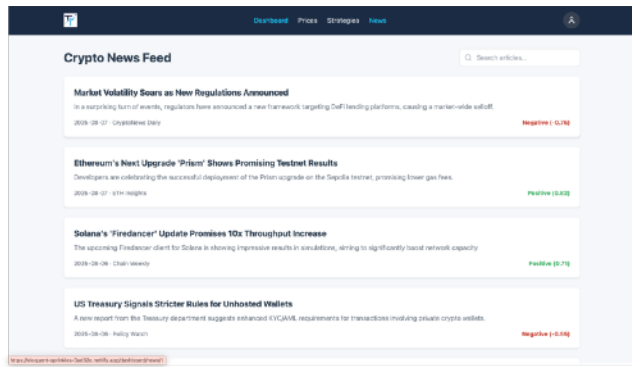
3D. Coin Prices Page



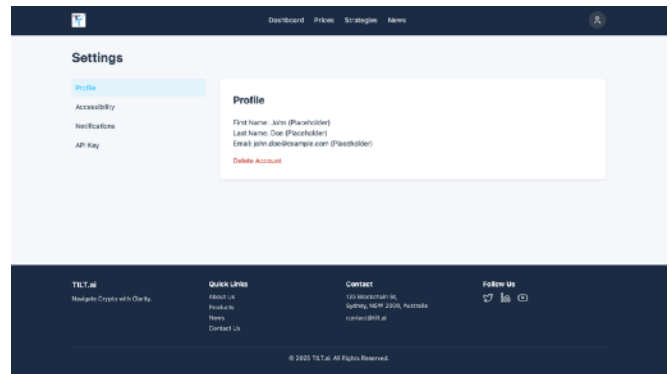
3E. Strategies Page



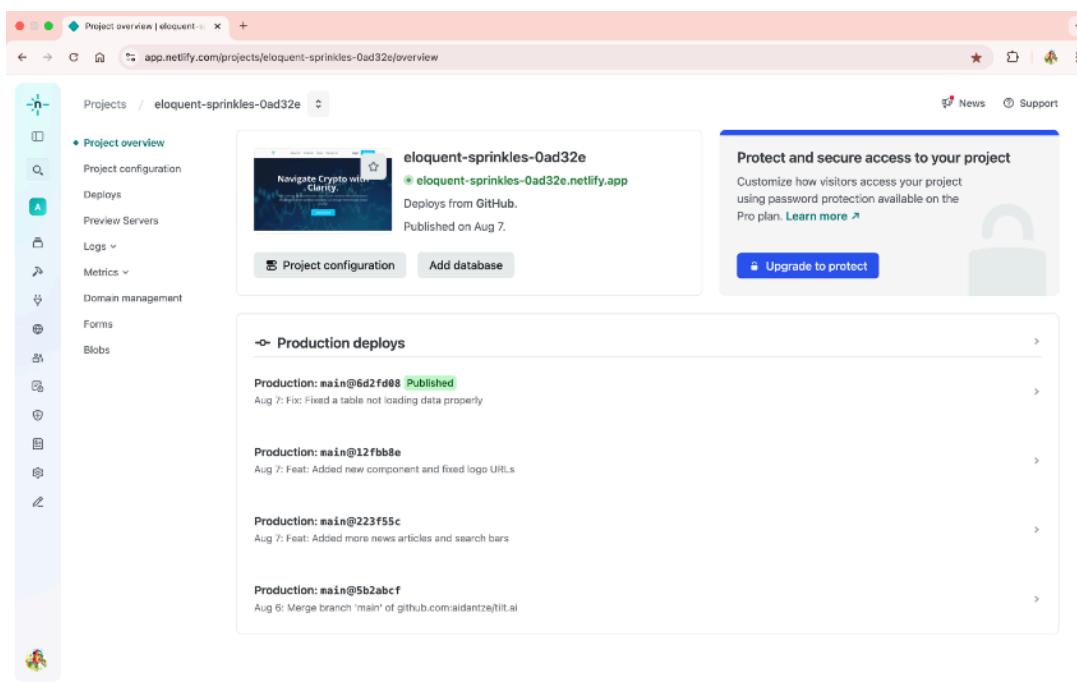
3F. News Page



3G. Settings Page



Appendix 4 – Screenshot of Netlify deployment dashboard



Appendix 5 – Prompt given to Gemini to build a working frontend

Build a working application for TILT.ai using React. This application should be a crypto investment platform with dashboards to visualise the data and recommend the right portfolio outcomes. I plan on deploying the service as a static website for now, as it will just be a static prototype.

The platform must be robust, intuitive and easy to use even for beginners in crypto. However, this is just a prototype static website using React.js and styling using either tailwind or bootstrap. It need not call any APIs. The only functionality should occur in the form of pressing buttons to navigate between pages. Images to be visualised can be access via cloundinary URLs. Any instance of the word “placeholder” means the buttons need not do anything, or the text provided must be fake information: you make it up!

The platform should consist of the following pages

1. Landing page [public]

- First page the user sees after navigating from google
- Introduce the page with an appropriate tagline and description (actually generate one as part of the prompt)
- Place an image on the right that can be used to draw the users interest.
- Modify the header to have the buttons for login/register (instead of the profile icon) and different pages to what can be seen in the dashboard:
 - About Us
 - Products
 - News (this feature can be accessed publicly)
 - Contact Us
- Different panels to showcase the full range of features as the user scrolls down the page
- Place the footer at the bottom

2. Login/Register page

- Tab at the top to navigate between login and register
- Login:
 - User enters email and password to log in
 - Error message is shown if password is incorrect
 - Link provided to allow the user to reset their password. This is placeholder
 - Integration with Google, Slack, Microsoft or Apple (passkeys). This is placeholder
 - Login button at the bottom should submit the login form and navigate to Dashboard, no matter what text is entered in (since this is placeholder, no security yet)
- Register:
 - User enters first name, last name, email and password

3. Dashboard page (overview) [only access after login]

- Upon first entry into this page after registration, user should be presented with a modal for personalisation:
 - 1. What would you like to use this platform for? Range of options as answers (e.g. work, professional, education, etc.) as placeholders. Clicking on any option navigates to the next question
 - 2. What are you interested in? Range of options as answers (e.g. invest in crypto, optimise portfolios, learn how to invest, other (please specify), etc.) as placeholders. Clicking on any option navigates to the next question
 - 3. Agreement of terms and conditions: link to T&Cs as placeholder, checkbox with text "I accept terms and conditions". Checking this box AND pressing "get started" button finally exits the modal
- Header at the top, footer on the bottom
- Range of different dashboard visualisations all on one page, organised in a neat grid layout rather than having them all underneath each other. For each metric show:
 - The title of the plot
 - Some placeholder text of the status of the coin/market price/sentiment
 - Button to allow the user to download the plot to their device (placeholder)
- Use the following titles and image URLs for the visualisations:

- Daily average sentiment line graph for top 5 coins: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445501/sentiment_daily_line_hllyws.png
 - Average sentiment bar graph for all coins VS market: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445485/mean_sentiment_all_vs_market_giulql.png
 - Fear-Greed bar graph for all coins VS market: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445477/fear_greed_bar_chart_gjjchv.png
 - Daily average sentiment line graph for market sentiment:
 - Fear-Greed Gauge for market sentiment: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445481/fear_greed_gauge_MARKET_ivzo9k.png
 - Cumulative returns for the top 5 coins: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445477/cumulative_returns_top5_coins_bvhgbr.png
 - Strategy VS buy-hold for top 5 coins: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445501/strategy_vs_buyhold_top5_coins_mecenz.png
 - Performance heat map for top 5 EW factors: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445486/performance_heatmap_top5_coins_dbpcr8.png
 - Performance metrics radar chart: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445486/performance_metrics_radar_strategy_jwbobz.png
 - Monthly return "candles" for top 5 EW factors: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445485/monthly_return_candles_top5_kpqzim.png
 - Rolling Volatility for top 5 coins: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445488/rolling_volatility_top5_k8w3id.png
 - Rolling Sharpe Ratio for top 5 coins: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445488/rolling_sharpe_ratio_top5_crhbuu.png
 - Return Distribution for top 5 coins: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445487/return_distribution_top5_gyxud9.png
 - Correlation matrix for top 5 coins: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445477/correlation_matrix_top5_ouhoxg.png
 - Use the following titles and generate tables based on the txt files provided as URLs:
 - Latest sentiment scores: https://res.cloudinary.com/dg6nuqapw/raw/upload/v1754450555/latest_sentiment_scores_xfxl05.txt
 - Coin performance ranking: https://res.cloudinary.com/dg6nuqapw/raw/upload/v1754450555/coin_performance_summary_qudq2a.txt
 - Include a + button on the top right to allow the user to add a new visualisation (placeholder)
 - Include a search button on the top right to search for a specific coin or strategy (placeholder)
 - On the bottom right corner, ensure to include a chatbot button which could act as a modal or a expanded dropdown to facilitate communication to the chatbot via a text box. The chatbot integration should not actually work functionally, but just to exist as proof of concept.
4. View specific coin prices page [only access after login]
- For now, visualise the coin prices for BTC.
 - Add a text box which doubles as a dropdown to search for coins. Dropdown is placeholder, with options as name and symbol of coin (e.g. Bitcoin BTC)
 - Use the following titles and image URLs for the visualisations:
 - Daily average sentiment line graph for BTC: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445491/sentiment_daily_line_BTC_acfga2.png
 - Fear-Greed Gauge for BTC sentiment: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445479/fear_greed_gauge_BTC_xujy7q.png

- Fear-Greed bar graph for BTC VS market: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754452890/fear_greed_bar_chart_btc_vs_market_y7n6lc.png
- Cumulative returns for BTC: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754452891/cumulative_return_btc_aw6jip.png
- Strategy VS buy-hold for BTC: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754452891/strategy_vs_buyhold_btc_xt0iw7.png
- Rolling Volatility for BTC: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754452891/rolling_volatility_btc_szuluq.png
- Rolling Sharpe Ratio for BTC: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754452891/rolling_sharpe_ratio_btc_gfhr6m.png
- Return Distribution for BTC: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754452891/return_distribution_btc_vvxi2h.png
- Include a + button on the top right to allow the user to add a new visualisation (placeholder)
- Include a search button on the top right to search for a specific coin or strategy (placeholder)

5. View strategies page [only access after login]

- Use the following titles and image URLs for the visualisations:
 - Cumulative returns for each strategy VS buy-hold (EW): https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445501/strategy_vs_buyhold_all_h7yi8v.png
 - Performance heat map for each strategy: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445486/performance_heatmap_strategies_anqlej.png
 - Performance metrics radar chart: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445486/performance_metrics_radar_strategy_jwbobz.png
 - Rolling Volatility for each portfolio strategy: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445488/rolling_volatility_strategies_x2yscm.png
 - Rolling Sharpe Ratio for each portfolio strategy: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445487/rolling_sharpe_ratio_strategies_hrgvml.png
 - Return Distribution for each strategy: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445487/return_distribution_all_strategies_vzf1s6.png
 - Correlation matrix for each strategy: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754445477/correlation_matrix_all_strategies_oxmqob.png
- Use the following titles and generate tables based on the txt files provided as URLs:
 - Strategy performance summary: https://res.cloudinary.com/dg6nuqapw/raw/upload/v1754450555/strategy_performance_summary_i4u1dm.txt
 - Current strategy signals: https://res.cloudinary.com/dg6nuqapw/raw/upload/v1754450554/current_strategy_signals_mhykkf.txt
- Include a + button on the top right to allow the user to add a new visualisation (placeholder)
- Include a search button on the top right to search for a specific coin or strategy (placeholder)

6. View all news page [public]

- Present all news articles in a list, each element of the list containing
 - News article title
 - First few characters of the content
 - Image of article if applicable, otherwise don't include one
 - Upload date
 - Source of the news (e.g. crypto compare)
 - Coins they relate to: comma-separated string (e.g. BTC, ETH, MARKET)
 - Sentiment score of the article

- Sentiment of the article based on score (positive, negative or neutral)
- All news articles in list should be placeholders (i.e. make it all up!)
- When user clicks on any article, navigate to View specific article contents

7. View specific article contents [public or private depending on the article to be viewed]

- Presents a single article
 - Title at the top
 - Below title, in smaller text, upload date, source of news, coins they relate to, sentiment score and sentiment text
 - Below that, image of article if applicable
 - Below that, full contents of the article
 - At the bottom, recommended list of other similar articles (limit to 6 results)
- All article elements and list should be placeholders (i.e. make it up)
- Download button on top right corner to allow the user to download the article as a PDF (placeholder)
- Back button on top left corner to navigate user back to View all news page

8. Settings page

- Sidebar to navigate to all the different settings options:
 - Profile:
 - Show the user's first and last name, email, and profile pic which they can edit. You must use fake information as placeholders.
 - Allow user to add a bio in a text box. This is a placeholder.
 - Allow user to integrate social media (e.g. LinkedIn) links., These are placeholders
 - Allow a button to reset password. This is a placeholder.
 - At the bottom have a red button to allow the user to delete their account, and a modal with a confirmation message for deletion prompting the user to enter the word CONFIRM in a text box before deletion, or to cancel. Once deleted, user is navigated back to Landing page. Again, this is a placeholder.
 - Accessibility:
 - Themes: light or dark mode (or system mode)
 - Grayscale
 - Larger text
 - Border elements
 - Dyslexia-friendly font
 - Ability to change the colours of tickers from green/red to red/green for asian-ethnic users. This feature should be disabled if grayscale is enabled.
 - All of these things should be placeholders, they should not do anything
 - Notifications Preferences
 - Checkboxes for different notifications preferences such as:
 - Notify when:
 - Sentiment for a coin is optimal
 - Price for a coin is optimal
 - Market sentiment is optimal
 - A different strategy predicts more optimal returns
 - etc.
 - Notify by:
 - Email

- SMS
- PagerDuty integration
- A button at the bottom to disable all notifications or reset notifications preferences
- All of these things should be placeholders, they should not do anything
- API Key
 - Button to copy API key. Placeholder button. DO NOT show the API key at all here!
 - That's all this section does!

Fundamental components

- Dashboard Header:
 - Containing the TILT.ai logo, the profile icon and the following pages:
 - Prices -> navigate user to View specific coin prices page
 - Strategies -> navigate user to View strategies page
 - News -> navigate user to View all news page
 - Any other necessary pages (these other navigation buttons shouldn't need to work, as I only plan on implementing the pages listed above)
 - Profile icon: dropdown to allow the user to access their account, settings and logout features
 - TILT.ai logo in header: user can click on this to return to the landing page at anytime
 - Logo with white background: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754450153/tilt.ai_logo_nytds8.png
 - Logo with transparent background: https://res.cloudinary.com/dg6nuqapw/image/upload/v1754450153/tilt.ai_logo_transparent_t6j0fy.png
- Footer:
 - Containing some other links to pages, some (fake) contact details and icons for social media (which also shouldn't work), as well as a (fake) copyright sentence

Design considerations

- Design for accessibility as well as visual appeal
- Ensure an appropriate amount of whitespace surrounding each element. That being said, the dashboard pages would likely have a lot of data on the page at once, so whitespace may need to be compromised
- Use the following colour palette (in hex values):
 - Primary Palette
 - Brand Blue: #1A2B43
 - A deep, serious blue to convey trust, stability, and professionalism, reminiscent of traditional finance.
 - Accent Cyan: #00C8FF
 - A vibrant, electric cyan for key interactive elements like buttons, highlights, and graphs. It signifies a forward-looking, tech-focused, and dynamic service.
 - UI & Typography
 - Background Grey: #F5F7FA
 - A soft, light grey to provide a clean and modern backdrop, improving readability and reducing eye strain.
 - Dark Text: #2C3E50
 - A deep, off-black for all text and UI elements to ensure high contrast and legibility against the light background.

- Sentiment & Data Visualization: simply display the images, no need to change colours for these plots. However, if there needed to visualise anything (e.g. tables), you can use the following colours:
 - Greed/Positive: #27AE60
 - A clear, confident green to represent positive market sentiment, gains, and "greed" periods.
 - Fear/Negative: #C0392B
 - A strong, bold red to signal negative market sentiment, losses, and "fear" periods.
 - Neutral: #F1C40F
 - A warm, neutral yellow to represent periods of low volatility or neutral market mood.